

CPSC 4200

Partners: Trent Brown and Ben Simmons

Assignment: Homework 1

Question 1:

Intro:

In this assignment, we explored the concept of a length extension attack on SHA-256. The objective was to modify the given authorized API URL by appending an additional command without knowing the secret 8-byte password. Our goal was to generate a new valid URL that would successfully execute the "UnlockSafes" command.

Steps:

1. We both started with analyzing the given API URL structure. This along with the attached video was necessary to understand how the token and command were formatted.

```
http://cpsc4200.mpese.com/username/lengthextension/api?token=token&command=command1&command=command2&...
```

2. We then identified the token from the original URL provided in the 'Water Lawn' linked. We determined the token was computed as the hash of an unknown 8-byte secret key concatenated with the original command string.
3. We next used the extracted token with SHA-256 to continue hashing as if we had started with the original input, including the secret password, even though we never actually knew the password. To perform this attack correctly, we had to account for the padding of the original message.
4. We calculated how the server would pad the original input including the secret, then crafted our extended message accordingly so that it aligned with the SHA-256 block structure
5. Next, we added the command &commandUnlockSafes to the hashed message. This ensured that the transition from the original message to our extended url was correct and seamless.
6. Finally, we computed the new hash from our modified input.

We did each seek assistance, both individually and collectively to better help understand how the attack was to take place. We reached out for help when trying to identify the original token. After watching the linked video, we realized that to generate the new URL, we needed to encode the original URL with:

- Our token
- The 8-byte secret password (which we did not know but could account for in padding)
- The new padding

- The new suffix (&command=UnlockSafes)

Additionally, we met with the TAs on Monday, January 27, and Wednesday, January 29. During these meetings, we discussed our approach to calculating the padding equation and ensuring that our attack properly extended the original message without breaking the hash structure.

From here on each partner worked collectively to complete these following steps:

Extracting Components:

- We parsed the given URL to extract the token and the command parameters.
- We identified the length of the original command sequence to compute the correct padding.

Setting Up the Length Extension Attack:

- We used the pysha256 module to initialize SHA-256 with the extracted token.
- We set the internal state of the hash function to the state derived from the original token.
- We adjusted the counter to reflect the padded length of the original message.

Appending the New Command:

- We computed the necessary padding to maintain SHA-256 integrity.
- We appended the new command &command=UnlockSafes
- We rehashed the modified message using the adjusted SHA-256 instance.

Generating the Final URL:

- We encoded the new hash and attached it to the modified URL.
- We verified the structure and correctness of the URL before testing.

Challenges and Debugging:

- Understanding the correct padding format required multiple attempts.
- Encoding issues arose when appending the new command.
- Adjusting the hash function's internal state correctly was crucial to generating a valid hash.
- Initially, we forgot to correctly append the padding before the new command, which led to invalid hashes.
- We verified our approach by using smaller test cases before applying it to the actual URL.

- Our Final issue we found was in our padded_len equation

```
orig_msg = url.suffix.encode()
secret_length = 8
orig_len = len(orig_msg) + secret_length

padded_len = orig_len + len(padding(orig_len))
```

Where Padded_len = m + padding(len(m))+x

Question 2:

Intro:

Question 2 we practiced and executed a hash collision attack. The goal of the assignment was to generate 2 different files evil.py and good.py that had identical MD5 hash but different output when the file is run. These files both had different outputs that were simply print statements, good.py printed "Use SHA-256 instead!" and evil.py printed "MD5 is perfectly secure!". Due to the hash collision attack method(e.g. prefix || blobA || suffix) we could have evil.py and good.py have identical hashes but different output. This is a vulnerability in the MD5 algorithm because of the length MD5 is subject too and also it being outdated is reason why we have algorithms like fastcoll that are able to generate files with identical hashes but different content.

Steps to a successful hash collision attack:

1. First we generated new files prefix and suffix:
 - a. Instead of using blob="" we used message="" this was to separate the code from the example in the hw, even tho changing blob to message does not have any effect it helped us understand a little better what we were doing
 - b. Suffix was the same we used sha256(message.encode.... Instead of sha256(blob.encode...
 - c. Suffix is also added to later in the steps
2. Next using the command line "fastcoll -p prefix -o col1 col2" we generated a hash collision in two different files col1& col2 that have identical MD5 hashes: here is a screenshot of doing so:

```
eecs388 → /work $ fastcoll -p prefix -o col1 col2
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'col1' and 'col2'
Using prefixfile: 'prefix'
Using initial value: c3f07b794359b40c73a93b065385a5b1

Generating first block: ...
Generating second block: W.....
Running time: 0.985266 s
eecs388 → /work $
```

a.

- b. Generating these files could take less than a second or up to around 10 seconds
3. After we got col1 and col2 we executed the command "cat col1 suffix > good.py; cat col2 suffix > evil.py". This command is one of the most important parts of the assignment. Lots is going on:
 - a. First we are appending the file col1 and suffix and then appending that to a new file good.py
 - b. Then we are appending col2 and suffix and then appending those files to a new file evil.py
 - c. This process is somewhat similar to a length extension attack because of the appending of different data to different files but due to a similar hash and suffix they are treated as the same in the MD5 algorithm
4. Now we have two files good.py and evil.py that have the same MD5 hash but we needed to have them generate different output and not just a sha256 hash of the message/blob. In order to do this we added a conditional to the end of our suffix file.
 - a. This conditional checks the hash of given and based on the hash prints a certain output:

```
eecs388 → /work $ python3 good.py
d676f856fb0357ca716c7bdc852ee08961510ab5119c9ed59afc209edf9a5564
```

We ran this before changing the suffix file to get a specific hash to the good.py file so that we can verify what output to print.

5. After getting the hash we created a conditional in the suffix file:

```
"""
from hashlib import sha256
if sha256(message.encode("latin-1")).hexdigest() == "d676f856fb0357ca716c7bdc852ee08961510ab5119c9ed59afc209edf9a5564":
    print("Use SHA-256 instead!")
else:
    print("MD5 is perfectly secure!")eecs388 → /work $
```

- a. This verified what hash was being used specifically, a sha256 hash and if it was from good.py then 'Use SHA-256 instead!' would be printed out
6. Now that suffix was changed the MD5 output would be the same for both good.py and evil.py and the output when you ran either would be different

```
eecs388 → /work $ python3 good.py
Use SHA-256 instead!
eecs388 → /work $ python3 evil.py
MD5 is perfectly secure!
eecs388 → /work $ openssl dgst -md5 good.py evil.py
MD5(good.py)= 892e3929dedd5e492a2c349c9e0d77c9
MD5(evil.py)= 892e3929dedd5e492a2c349c9e0d77c9
a. eecs388 → /work $
```

Road Blocks:

- We ran into a road block when it came to deciding when to print the statements:
 - Questions we had were:

- Where do we put the print statements?
- Do we include them before or after we create the hash collision?
- Are they imbedded in the good and evil python files?
- After a meeting with the TA, we understood better what we needed to do and this was including the conditional into the suffix file **AFTER** we get the SHA-256 hash from running python3 good.py
- This is was the main challenge of question 2 because we understood the example before hand, however adding the print statements created the challenge. After, numerous attempts of running through the example and implementing the statements and talking with the TA it came clear to us how to apply the Hash Collision attack succefully.